Programming and Engineering Challenge (5EWC0)
Formative exam 28-10-2025: Banking

In this formative exam you will develop a program that can be used by a bank to keep track of the balance on the bank accounts of its customers. The program supports commands to add or withdraw funds from accounts. It can also be used to bill customers a service charge for having their account at the bank. Interest payments on debts and credit are also managed through the program. In addition, the bank can use the program to compute the percentage of debtors among its account holders.

Hint: In this formative exam, you get a basic skeleton program that already compiles. When the program is started, the user is greeted by printing the following text to the terminal:

```
** Bank **
```

The skeleton program already responds to a command to quit the program when the user inputs the character 'q'. This command terminates the program, and it outputs the value of the two variables named `accounts` and `bank`, as declared in the `main` function. This is done using the function `outputAccountData`. In addition, the `main` function contains a call to the function `initializeAccountData`. You should leave these two functions and their calls in the `main` function unchanged. When the user inputs the character 'q' as a command, the output of the program will look as follows:

```
** Bank **
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 0.00
Accounts 4: -50.45
** bank **
The first PEC bank [v1.0]
```

The skeleton program already passes test case 0. This test case verifies that your changes in the subsequent tasks do not break the functionality of the skeleton program.

Task 1. In this task, you will implement a command to print for each account in the array `accounts` whether the account has a positive or negative balance. Accounts with a positive balance have a value equal to, or larger than 0. Accounts with a negative balance have a value strictly smaller than 0.

Write a function `void printPosNeg(float accounts[], int size)` that prints for each account in the array `accounts` whether the account has a positive or negative balance. The number of accounts in the array is stored in the variable `size`. Extend also the `main` function such that when the user inputs the character 'p' as command, the function `printPosNeg` is called with the array `accounts` declared in the `main` function and the size of the array.

The output of your program should look as follows:

```
** Bank **
Command? p
Account 0: negative
Account 1: positive
Account 2: positive
Account 3: positive
Account 4: negative
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 0.00
Accounts 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Test cases related to this task: case 1

Task 2. Write a function `void serviceCharge(float accounts[], int size, float charge)` that deducts a service charge from all accounts in the array `accounts`. The variable `charge` contains the amount that needs to be deducted from each account in the array `accounts`. The number of accounts in the array is stored in the variable `size`.

Extend also the `main` function such that when the user inputs the character 's' as command, the user is asked to input the amount to be charged as service charge to all accounts. Subsequently, the function `serviceCharge` is called with this value and the array `accounts` declared in the `main` function. The output of your program will now look as follows:

```
** Bank **
Command? s
Service charge? 2.34
Command? q
** accounts **
Accounts 0: -102.34
Accounts 1: 2.31
Accounts 2: 1330.99
Accounts 3: -2.34
Accounts 4: -52.79
** bank **
The first PEC bank [v1.0]
```

Test cases related to this task: case 2

Task 3. Write a function `void interestAccounts(float accounts[], int size)` that adds interest to each account in the array `accounts`. The table below shows the interest rate per tier. Accounts with a negative balance need to pay 5% interest to the bank. Accounts with a balance from 0.00 up-to and including 1000.00 receive 2% interest. Accounts with a balance above 1000.00 receive 2% interest over their first 1000.00 and 1% interest over the remainder. So, an account with a balance of 1500.00 will therefore receive 2% interest over the first 1000.00 and 1% interest over the remaining 500.00.

amount	interest rate
below 0.00	5%
from 0.00 up-to 1000.00	2%
above 1000.00	1%

Extend also the `main` function such that when the user inputs the character 'i' as command, the function `interestAccounts` is called with the array `accounts` declared in the `main` function.

The output of your program will now look as follows:

```
** Bank **
Command? i
Command? q
** accounts **
Accounts 0: -105.00
Accounts 1: 4.74
Accounts 2: 1356.66
Accounts 3: 0.00
Accounts 4: -52.97
** bank **
The first PEC bank [v1.0]
```

Test cases related to this task: case 3

Task 4. Write a function `void changeBalance(float accounts[], int id, float amount)` that adds the value of the variable `amount` to the value stored in index `id` of the array `accounts`.

Extend also the `main` function such that when the user inputs the character 'b' as command, the user is asked to input the id of the account and the amount that needs to be added to this account. The program should check that the inputted id is within the bounds of the array. If an id is input that is outside the bounds, then the program should ask the user for another id till an id within the bounds of the array is input. Subsequently the function `changeBalance` should be called with the array `accounts` declared in the `main` function.

The output of your program should now look as follows:

```
** Bank **
Command? b
Account? 15
Account? -1
Account? 3
Amount? 100
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 100.00
Accounts 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Test cases related to this task: case 4-5

Task 5. Write a function `float percentageDebtors(float accounts[], int size))` that computes the percentage of accounts that have a negative balance in the array `accounts` to the total number of accounts, `size`, in the array `accounts`. Hence, the function returns a value between 0.0 and 100.0.

Extend also the `main` function such that when the user inputs the command 'd', the program calls the `percentageDebtors` function with the array `accounts` declared in the `main` function. Subsequently the percentage of accounts that have a debt is printed as a percentage (between 0.0% and 100.0%) with one decimal precision to the terminal.

The output of your program should now look as follows:

```
** Bank **
Command? d
Percentage debtors: 40.0
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 0.00
Accounts 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Test cases related to this task: case 6

Task 6. Write a function `void removeVersion(char str[])` that removes the brackets '[' and ']' and any text in between them from the string `str`. You may assume that `str` contains exactly one pair of brackets '[' and ']'.

Extend also the `main` function such that when the user inputs the command 'r', the program calls the `removeVersion` function with the array `bank` declared in the `main` function.

The output of your program should now look as follows:

```
** Bank **
Command? r
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 0.00
Accounts 4: -50.45
** bank **
The first PEC bank
```

Test cases related to this task: case 7

Task 7. Extend the `main` function such that when the user inputs the command 'n', the program asks the user to input a string of up to 63 characters terminated with a newline ('\n') character. The returned string, without the newline character, should be stored in the variable `bank` declared in the `main` function.

The output of your program should now look as follows:

```
** Bank **
Command? n
Bank name? The second PEC bank [sw v1.2.3]
Command? q
** accounts **
Accounts 0: -100.00
Accounts 1: 4.65
Accounts 2: 1333.33
Accounts 3: 0.00
Accounts 4: -50.45
** bank **
The second PEC bank [sw v1.2.3]
```

Test cases related to this task: case 8-9

Hint: Test case 9 covers all previous tasks. Failing test case 9 might therefore imply that you still have a bug in an earlier task.

Input / output test cases

Case 0

Input:

```
q
```

Output:

```
** Bank **
Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Case 1

Input:

```
p
q
```

Output:

```
** Bank **
Command? Account 0: negative
Account 1: positive
Account 2: positive
Account 3: positive
Account 4: negative
Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Case 2

Input:

```
s
2.34
q
```

Output:

```
** Bank **
Command? Service charge? Command? ** accounts **
Account 0: -102.34
Account 1: 2.31
Account 2: 1330.99
Account 3: -2.34
Account 4: -52.79
** bank **
The first PEC bank [v1.0]
```

Case 3

Input:

```
i
q
```

Output:

```
** Bank **
Command? Command? ** accounts **
Account 0: -105.00
Account 1: 4.74
Account 2: 1356.66
Account 3: 0.00
Account 4: -52.97
** bank **
The first PEC bank [v1.0]
```

Case 4

Input:

```
b
0
-23.45
b
4
123.99
q
```



Output:

```
** Bank **
Command? Account? Amount? Command? Account? Amount? Command? ** accounts **
Account 0: -123.45
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: 73.54
** bank **
The first PEC bank [v1.0]
```

Case 5

Input:

```
b
-1
5
3
-12.34
q
```

Output:

```
** Bank **
Command? Account? Account? Account? Amount? Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: -12.34
Account 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Case 6

Input:

```
d
q
```

Output:

```
** Bank **
Command? Percentage debtors: 40.0
Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: -50.45
** bank **
The first PEC bank [v1.0]
```

Case 7

Input:

```
r
q
```

Output:

```
** Bank **
Command? Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: -50.45
** bank **
The first PEC bank
```

Case 8

Input:

```
n
The second PEC bank [sw v1.2.3]
q
```

Output:

```
** Bank **
Command? Bank name? Command? ** accounts **
Account 0: -100.00
Account 1: 4.65
Account 2: 1333.33
Account 3: 0.00
Account 4: -50.45
** bank **
The second PEC bank [sw v1.2.3]
```



Case 9

Input:

```
s
4.45
b
2
-328.88
b
3
-5000
i
p
d
n
The third PEC bank [version (1.2a)]
r
q
```

Output:

```
** Bank **
Command? Service charge? Command? Account? Amount? Command? Account? Amount?
Command? Command? Account 0: negative
Account 1: positive
Account 2: positive
Account 3: negative
Account 4: negative
Command? Percentage debtors: 60.0
Command? Bank name? Command? Command? ** accounts **
Account 0: -109.67
Account 1: 0.20
Account 2: 1020.00
Account 3: -5254.67
Account 4: -57.65
** bank **
The third PEC bank
```